

C++ Learning Projects

Especificaciones para A Tour of C++ — Bjarne Stroustrup (3ª ed.)

6 proyectos · C++20 · Perfil: Java → C++

Proyecto 1 — Calculadora de expresiones

Capítulo trigger	Cap. 1 (The Basics)
Dificultad	Baja
Tiempo estimado	4–6 horas
Compilador	g++ -std=c++20 -Wall -Wextra

Descripción

Implementa una calculadora de consola que evalúe expresiones aritméticas simples introducidas por el usuario línea a línea. No se usan clases. El objetivo es forzar el uso consciente de tipos, punteros, referencias y funciones tal como los entiende C++, no Java.

REQUISITOS FUNCIONALES

- El programa lee una expresión por línea desde stdin hasta que el usuario escribe 'q' o 'quit'.
- Soporta operaciones: suma (+), resta (-), multiplicación (*), división (/), módulo (%).
- Soporta operandos enteros (int) y decimales (double). Detecta y usa el tipo correcto.
- Muestra el resultado con formato: '>> 42' o '>> 3.14'.
- Si la expresión es inválida (división por cero, formato incorrecto), muestra un mensaje de error claro y continúa sin terminar el programa.
- Implementa un historial: el comando 'hist' muestra las últimas 10 operaciones con sus resultados.

REQUISITOS TECNICOS (lo que debes usar)

- Mínimo 4 funciones separadas: lectura, parseo, cálculo y presentación.
- Al menos un parámetro pasado por referencia y al menos uno por puntero (justifica en comentario por qué usas cada uno).
- Usa const correctamente: cualquier parámetro que no se modifique debe ser const.
- El historial se almacena en un array de tamaño fijo (no std::vector todavía).
- No se permite usar std::string::find o regex; parsea la expresión manualmente.

CRITERIOS DE ÉXITO

- Compila sin warnings con -Wall -Wextra.
- Puedes explicar en voz alta la diferencia entre pasar por valor, por referencia y por puntero en cada función que lo usa.
- El programa no termina inesperadamente con ninguna entrada del usuario.

PISTA CLAVE

En Java los tipos primitivos se pasan por valor y los objetos por referencia automáticamente. En C++ todo se pasa por valor por defecto. Reflexiona sobre qué pasa con la memoria cuando llamas a una función.

Proyecto 2 — Gestor de contactos con struct

Capítulo trigger	Cap. 2 (User-Defined Types)
Dificultad	Baja–Media
Tiempo estimado	5–7 horas
Compilador	g++ -std=c++20 -Wall -Wextra

Descripción

Construye un sistema de agenda de contactos usando struct, enum y union, sin clases. La idea es entender cómo C++ define tipos antes de llegar a OOP, y compararlo conscientemente con cómo lo harías con una clase en Java.

REQUISITOS FUNCIONALES

- Define un struct Contacto con: nombre (char[50]), teléfono (char[20]), email (char[60]), tipo (enum: PERSONAL, TRABAJO, FAMILIA, OTRO).
- Capacidad para almacenar hasta 100 contactos en un array estático.
- Menú de consola con operaciones: añadir, buscar por nombre, listar todos, eliminar por nombre, salir.
- La búsqueda es case-insensitive.
- Al listar, muestra los contactos agrupados por tipo (todos los TRABAJO juntos, etc.).
- Usa un union para demostrar su funcionamiento: el contacto puede tener 'teléfono' o 'extensión de empresa' (int), nunca los dos a la vez. Muestra el campo correcto según el tipo.

REQUISITOS TÉCNICOS

- El struct Contacto no puede contener métodos (eso lo reservamos para el cap. 5).
- Todas las operaciones sobre contactos deben ser funciones libres que reciben el array y su tamaño.
- Usa enum class (no enum a secas) para el tipo de contacto.
- Implementa al menos una función que devuelva un puntero a Contacto (para buscar).
- Documenta con un comentario corto cada función explicando qué hace y por qué su firma es así.

CRITERIOS DE ÉXITO

- Puedes explicar la diferencia entre struct en C++ y class en Java.
- Entiendes por qué union es útil y cuándo es peligroso.
- Puedes explicar qué ventaja aporta enum class frente a enum en C.

PISTA CLAVE

Un struct en C++ es casi idéntico a una clase con todo public. La diferencia conceptual llegará en el cap. 5. Por ahora, céntrate en entender cómo C++ gestiona tipos de datos compuestos sin el envoltorio de un objeto.

Proyecto 3 — Librería de matemáticas propia

Capítulo trigger	Cap. 3 (Modularity)
Dificultad	Media
Tiempo estimado	5–8 horas
Compilador	g++ -std=c++20 -Wall -Wextra (múltiples archivos)

Descripción

Refactoriza la calculadora del Proyecto 1 en una librería de múltiples archivos con namespaces. Aprenderás compilación separada, organización de cabeceras y la diferencia entre declaración y definición, algo que en Java no existe de la misma forma.

ESTRUCTURA DE ARCHIVOS REQUERIDA

- mathlib/basic.h + basic.cpp — operaciones aritméticas básicas
- mathlib/stats.h + stats.cpp — media, mediana, desviación estándar
- mathlib/trig.h + trig.cpp — sin, cos, tan con conversión grados↔radianes
- main.cpp — programa principal que usa la librería
- Makefile — compila todo con las dependencias correctas

REQUISITOS FUNCIONALES

- Namespace mathlib con sub-namespaces: mathlib::basic, mathlib::stats, mathlib::trig.
- mathlib::stats acepta un array de doubles y su tamaño, y calcula media, mediana y desviación estándar.
- mathlib::trig tiene funciones que trabajan en grados (no radianes) por defecto.
- El main.cpp usa using namespace solo en el ámbito local de funciones, nunca a nivel global.
- Cada .h tiene include guards (#ifndef / #define / #endif).

REQUISITOS TÉCNICOS

- Ningún .cpp incluye otro .cpp, solo sus propios .h y de la std.
- Las funciones de stats.cpp no dependen de nada de trig.cpp ni basic.cpp (acoplamiento cero).
- El Makefile compila cada .cpp en su propio .o y luego enlaza. No compila todo de golpe.
- Usa argumentos de función por valor cuando los datos son pequeños (int, double) y por referencia const cuando son arrays.

CRITERIOS DE ÉXITO

- Si cambias solo basic.cpp, el Makefile solo recompila basic.o y vuelve a enlazar.
- Puedes explicar por qué los .h solo tienen declaraciones y los .cpp las definiciones.
- Puedes explicar qué pasaría sin los include guards.

PISTA CLAVE

En Java el compilador maneja las dependencias automáticamente. En C++ tú eres responsable de qué ve el compilador en cada unidad de compilación. Esta separación manual es la base de por qué C++ puede compilar proyectos gigantes en paralelo.

Proyecto 4 — Parser de fichero CSV

Capítulo trigger	Cap. 4 (Error Handling)
Dificultad	Media
Tiempo estimado	6–8 horas
Compilador	g++ -std=c++20 -Wall -Wextra

Descripción

Construye un parser que lea ficheros CSV, valide su estructura y lance excepciones descriptivas cuando algo falla. El objetivo es dominar el sistema de excepciones de C++ y aprender a diseñar contratos de función explícitos.

REQUISITOS FUNCIONALES

- Lee cualquier fichero CSV pasado como argumento en `argv[]`.
- Primera fila = cabecera. Valida que todas las filas tengan el mismo número de columnas que la cabecera.
- Permite consultar el valor de una celda por (fila, columna) o por (fila, nombre_columna).
- Exporta a un fichero de salida filtrando solo las columnas que el usuario especifique.
- Modo estadístico: si una columna es numérica, calcula y muestra media, min y max.

REQUISITOS TÉCNICOS — EXCEPCIONES

- Define una jerarquía propia de excepciones heredando de `std::exception`: `CSVException` → `FileNotFoundException`, `MalformedRowException`, `ColumnNotFoundException`.
- Cada excepción incluye el número de línea o nombre de columna donde ocurrió el error.
- El parser usa `assert()` para invariantes internos (precondiciones que nunca deberían fallar si tu código es correcto).
- El `main()` tiene un bloque `try-catch` que captura cada tipo de excepción por separado y muestra mensajes diferentes.
- Nunca usas `catch(...)` como única captura (solo como último recurso con `re-throw`).

CRITERIOS DE ÉXITO

- Puedes explicar la diferencia entre una excepción y un `assert` en C++.
- El programa nunca termina con una excepción no capturada (`std::terminate`) en uso normal.
- Puedes explicar qué es RAII y por qué el fichero siempre se cierra aunque se lance una excepción.

PISTA CLAVE

Java tiene checked exceptions y unchecked exceptions. C++ solo tiene unchecked. Eso significa que el compilador no te avisa de qué puede lanzar una función; tú decides qué capturar y dónde. Diseña tu jerarquía de excepciones antes de empezar a codificar.

Proyecto 5 — Sistema de figuras geométricas

Capítulo trigger	Cap. 5 (Classes)
Dificultad	Media–Alta
Tiempo estimado	8–12 horas
Compilador	g++ -std=c++20 -Wall -Wextra

Descripción

Diseña una jerarquía de clases para figuras geométricas con tipos concretos y abstractos, funciones virtuales y herencia. Este es el proyecto donde C++ y Java divergen más visiblemente: aquí entiendes por qué C++ necesita virtual, qué es el object slicing y para qué sirven los destructores virtuales.

JERARQUÍA REQUERIDA

- Figura (clase abstracta base): `area()`, `perimetro()`, `describe()`, `nombre()` — todos virtuales puros.
- Circulo : public Figura — `radio` como miembro privado.
- Rectangulo : public Figura — `ancho` y `alto` privados.
- Triangulo : public Figura — tres lados privados, valida la desigualdad triangular en el constructor.
- FiguraCompuesta : public Figura — contiene un vector de `Figura*` y suma áreas y perímetros.

REQUISITOS FUNCIONALES

- Un programa principal que crea un `vector<Figura*>` con figuras mezcladas y las procesa polimórficamente.
- Función libre: `void imprimir_informe(const std::vector<Figura*>&)` que imprime todas las figuras ordenadas por área de menor a mayor.
- Función libre: `Figura* figura_mayor_area(const std::vector<Figura*>&)`.
- El usuario puede crear figuras interactivamente desde consola y añadirlas al vector.

REQUISITOS TÉCNICOS

- El destructor de `Figura` es virtual. Documenta con un comentario por qué es obligatorio aquí.
- Ninguna clase hija tiene miembros públicos (todo `private` o `protected` con getters si es necesario).
- Implementa el constructor de copia en al menos una clase hija y documenta qué hace el compilador si no lo defines.
- Demuestra object slicing: escribe un ejemplo comentado donde ocurre y explica por qué es un problema.
- No uses `dynamic_cast` en ningún sitio (si lo necesitas, hay un fallo de diseño).

CRITERIOS DE ÉXITO

- Puedes explicar por qué sin virtual en el destructor base se produce undefined behavior.
- Puedes explicar qué es object slicing y cómo prevenirlo con punteros o referencias.
- Puedes explicar la diferencia entre override y virtual en una función hija.

PISTA CLAVE

En Java todos los métodos son virtuales por defecto. En C++ no: una función no es polimórfica hasta que pones virtual. Esa decisión de diseño existe por razones de rendimiento. Reflexiona sobre el coste que tiene la vtable.

Proyecto 6 — Contenedor dinámico propio

Capítulo trigger	Cap. 6 (Essential Operations)
Dificultad	Alta
Tiempo estimado	10–15 horas
Compilador	g++ -std=c++20 -Wall -Wextra -fsanitize=address

Descripción

Implementa tu propio `MyVector<T>`, un contenedor dinámico genérico similar a `std::vector`. Este es el proyecto más importante de la ruta: entenderás RAII, los constructores especiales, move semantics y smart pointers desde dentro. Lo que la JVM hace por ti en Java, aquí lo haces tú a mano.

REQUISITOS FUNCIONALES

- `MyVector<T>` soporta `push_back(T)`, `pop_back()`, `operator[]`, `at(size_t)` con bounds checking.
- `size()` devuelve el número de elementos, `capacity()` la capacidad actual reservada.
- Crece automáticamente cuando se llena (duplica capacidad). Documenta la estrategia de crecimiento.
- `clear()` elimina todos los elementos sin liberar la memoria reservada.
- Iteradores básicos: `begin()`, `end()` que permiten usar `MyVector` en un range-for loop.

REQUISITOS TÉCNICOS — REGLA DE 5

- Implementa los 5 constructores especiales: default, copy, move, copy assignment, move assignment.
- El destructor libera correctamente la memoria. Compila con `-fsanitize=address` y verifica cero leaks.
- La memoria interna se gestiona con `unique_ptr<T[]>` (no raw pointer).
- El constructor de copia hace una deep copy. El de move deja el origen en estado válido pero vacío.
- Documenta con un comentario en cada constructor especial qué hace exactamente y cuándo lo llama el compilador.

EJERCICIOS DE VERIFICACIÓN

- Escribe un test que demuestre que mover un `MyVector` es más rápido que copiarlo (mide con `chrono`).
- Demuestra que `operator=` hace deep copy comparando las direcciones de memoria de ambos vectores.
- Crea un `MyVector<MyVector<int>>` (vector de vectores) y verifica que funciona correctamente.

CRITERIOS DE ÉXITO

- Cero leaks con `AddressSanitizer` en todos los casos de uso.
- Puedes explicar la diferencia entre copy semantics y move semantics con tu propio código como ejemplo.
- Puedes explicar qué es RAII y por qué `unique_ptr` es la herramienta correcta aquí.
- Puedes explicar qué hace el compilador si no defines el constructor de move.

PISTA CLAVE

En Java el garbage collector se encarga de liberar memoria. En C++ con RAII, la memoria se libera cuando el objeto sale de ámbito: el destructor es tu garbage collector, pero predecible y sin pausas. `unique_ptr` implementa RAII para ti; en este proyecto lo usas para entenderlo por dentro.

Proyecto 7 — Agenda de tareas con persistencia

Capítulo trigger	Cap. 7–9 (STL Containers, Algorithms, Utilities)
Dificultad	Media
Tiempo estimado	8–10 horas
Compilador	g++ -std=c++20 -Wall -Wextra

Descripción

Construye una agenda de tareas completa usando los contenedores y algoritmos de la STL. El objetivo es ver la STL en acción y contrastarla con Java Collections: son filosofías distintas (la STL separa contenedores de algoritmos, Java los une en la interfaz del objeto).

REQUISITOS FUNCIONALES

- Cada tarea tiene: id (int), título (string), descripción, prioridad (enum: BAJA, MEDIA, ALTA, CRÍTICA), etiquetas (conjunto de strings), fecha límite (opcional).
- Operaciones: añadir, completar, eliminar, buscar por título, filtrar por prioridad, filtrar por etiqueta.
- Las tareas se guardan en disco al salir y se cargan al arrancar (formato propio, no JSON).
- Muestra estadísticas: total de tareas, completadas, pendientes por prioridad.

REQUISITOS TECNICOS — CONTENEDORES STL

- `std::vector<Tarea>` como contenedor principal.
- `std::map<int, Tarea*>` como índice por id para acceso $O(\log n)$.
- `std::set<string>` para las etiquetas de cada tarea (sin duplicados, ordenado).
- `std::optional<FechaLimite>` para la fecha límite (puede no existir).

REQUISITOS TÉCNICOS — ALGORITMOS STL

- `std::sort` con comparador lambda para ordenar por prioridad.
- `std::find_if` para buscar tareas por predicado.
- `std::count_if` para las estadísticas.
- `std::transform` para extraer solo los títulos de un subconjunto.
- Al menos un uso de `std::ranges::` (`ranges::sort`, `ranges::filter`, o similar).

CRITERIOS DE ÉXITO

- Puedes explicar cuándo usar vector vs map vs set y qué complejidad tiene cada operación.
- Puedes explicar por qué en C++ los algoritmos están separados de los contenedores.
- Puedes explicar qué es un lambda y cómo captura variables del entorno.

PISTA CLAVE

En Java, `sort()` está en el objeto (`list.sort()`). En C++, `std::sort(begin, end)` es una función libre. Eso permite aplicar el mismo algoritmo a cualquier estructura que tenga iteradores, incluyendo las tuyas propias.

Proyecto 8 — Mini shell (proyecto integrador)

Capítulo trigger	Cap. 10–12 (Concepts, Utilities, Numerics)
Dificultad	Alta
Tiempo estimado	12–20 horas
Compilador	<code>g++ -std=c++20 -Wall -Wextra</code>

Descripción

Construye un intérprete de comandos de consola extensible. Es el proyecto integrador que usa todo lo aprendido más las características de C++20: `concepts`, `ranges` y `std::optional`. Su diseño modular te prepara exactamente para el tipo de código que verás en la asignatura del año que viene.

REQUISITOS FUNCIONALES

- Lee comandos línea a línea desde `stdin` con un prompt personalizable.
- Comandos built-in: `help`, `history`, `clear`, `echo`, `calc` (llama a tu librería del P3), `exit`.
- Sistema de plugins: cualquier función con la firma correcta se puede registrar como comando nuevo.
- Soporte de pipes básico: `comando1 | comando2` pasa la salida del primero como entrada del segundo.
- Historial de comandos navegable con flechas (si el sistema lo soporta) o con `!N` para ejecutar el comando N del historial.

REQUISITOS TÉCNICOS — C++20

- Define un `concept Comando` que restrinja qué tipos pueden registrarse como comandos del shell.
- Usa `std::optional<string>` para el resultado de ejecutar un comando (puede no producir output).
- El historial se implementa con `ranges::views::take` y `ranges::views::reverse` para mostrar los últimos N comandos.
- Usa `std::variant<BuiltinCmd, PluginCmd>` para representar los dos tipos de comandos.
- Al menos una función template con un `concept` como restricción.

ARQUITECTURA RECOMENDADA

- `shell/parser.h+cpp` — tokeniza la línea de entrada
- `shell/registry.h+cpp` — mapa de nombre → función comando
- `shell/executor.h+cpp` — ejecuta comandos y gestiona pipes
- `shell/history.h+cpp` — historial con `ranges`
- `main.cpp` — bucle principal REPL (Read-Eval-Print Loop)

CRITERIOS DE ÉXITO

- Puedes añadir un comando nuevo en menos de 10 líneas sin tocar el núcleo del shell.
- Puedes explicar qué ventaja aportan los concepts frente a los templates sin restricción.
- Puedes explicar qué es `std::variant` y por qué es mejor que una jerarquía de clases en este caso.
- El proyecto compila limpio y puedes presentarlo como ejemplo de código C++20 real.

PISTA CLAVE

Este proyecto no tiene una solución única. Hay múltiples decisiones de diseño válidas. El objetivo no es que funcione perfectamente, sino que cuando llegues a la asignatura puedas leer código C++20 sin que te resulte extraño y tengas criterio para tomar decisiones de diseño.

RESUMEN Progresión completa de la ruta

#	Proyecto	Fase	Horas est.	Concepto clave
1	Calculadora de expresiones	1 — Fundamentos	4–6 h	Punteros · referencias · const
2	Gestor de contactos	1 — Fundamentos	5–7 h	struct · enum class · union
3	Librería de matemáticas	2 — Modularidad	5–8 h	Namespaces · compilación separada
4	Parser CSV	2 — Errores	6–8 h	Excepciones · assert · RAII
5	Figuras geométricas	3 — OOP	8–12 h	virtual · polimorfismo · slicing
6	Contenedor dinámico MyVector	3 — Memoria	10–15 h	Regla de 5 · move · unique_ptr
7	Agenda de tareas	4 — STL	8–10 h	Contenedores · algoritmos · ranges
8	Mini shell	4 — C++20	12–20 h	Concepts · variant · optional

Total estimado: 58–86 horas · ~2–3 meses a ritmo de estudiante